Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Erişim şekline göre veri yapıları: dizi (rastgele erişim), bağlı liste (sıralı erişim), yığın (stack).
- Yığın (stack) ve LIFO (last in first out) mantığı: undo/redo, tarayıcıda geri gitme, fonksiyon çağrıları.
- Dizi ile yığın gerçekleştirme: `push`, `pop`, `isEmpty`, `isFull`, `peek`.
- Yığın uygulaması: aritmetik ifade değerlendirme (expression evaluation).
  - İki yığınla infix ifadeyi doğrudan çözme.
  - Infix ifadeyi postfix'e çevirme.
  - Postfix ifadeyi çözme.
- Kuyruk (queue) ve FIFO (first in first out) mantığı.
- Basit kuyruk, dairesel kuyruk (circular queue) ve bağlı liste ile kuyruk.
- Soru: Yığındaki en büyük elemanı her an bilmek (max stack).

Hocanın Özellikle Vurguladığı Kısımlar

- Yığında yalnız en üstteki elemana erişilebilir. Aradaki elemanlara erişmek yasaktır; yapının anlamı budur.
- Bu derste gerçekleştirilen yığın, donanımdaki stack bölgesi değil, dizi kullanılarak yazılan yüksek seviyeli (high level) bir yapıdır.
- `pop` öncesi yığının boş, `push` öncesi dolu olup olmadığı mutlaka kontrol edilmeli. Bu kontroller atlanırsa program yanlış çalışır.
- Eleman sayısı yaklaşık biliniyorsa yığın için dizi yeterlidir. Bağlı liste de kullanılabilir.
- Basit kuyrukta eleman çıkarmak $O(n)$'dir, çünkü tüm elemanlar bir öne kaydırılır. Dairesel kuyruk ve bağlı liste ile kuyruk bu sorunu çözer.
- `peek` ve dairesel kuyruğun `dequeue` fonksiyonu öğrencilere bırakıldı. İnternete bakmadan yazmaya çalışın.
- Hoca derste soru sorulmamasından ve online katılımın düşük olmasından şikâyet etti (yaklaşık 90 kayıtlı öğrenciden 22 kişi el kaldırdı). Uzaktan yapılan sınavların daha zor olmak zorunda olduğunu hatırlattı.
- Bu hafta bağlı listelerle ilgili ödev verilecek.

Kısa Tekrar Notları

- Yığın: LIFO. Son giren ilk çıkar.
- Kuyruk: FIFO. İlk giren ilk çıkar.
- Yığın işlemleri: `push` (ekle), `pop` (çıkar), `peek` (çıkarmadan en üsttekine bak), `isEmpty`, `isFull`.
- Bu derste `top` başlangıçta 0. `push`: `items[top] = value; top++`. `pop`: `top--; value = items[top]`. Boş: `top == 0`. Dolu: `top == MAX`.
- Infix'i çözmek için iki yığın kullanılır: biri işlenenler (operand), biri işlemciler (operator).
- Infix'ten postfix'e çevirmede yalnız işlemci yığını kullanılır. Postfix'i çözmede yalnız işlenen yığını kullanılır.
- Yığından iki işlenen çekildiğinde işlem "alttaki (işlem) üstteki" şeklinde yapılır.
- `3 * (5 + 8 / 2) - 4` ifadesinin postfix'i: `3 5 8 2 / + * 4 -`. Sonuç 23.
- Kuyruk işlemleri: `enqueue` (sona ekle), `dequeue` (baştan çıkar).

- Dairesel kuyrukta `rear = (rear + 1) % MAX`. enqueue ve dequeue $O(1)$.
- Bağlı liste ile kuyruk: `front` ve `rear` pointer'ları. Ekleme `rear`'dan, çıkarma `front`'tan; ikisi de $O(1)$.
- Max stack: En büyük değerleri tutan ikinci bir yığın kullanılır.

Detaylı Açıklamalar

Erişim Şekilleri

- **Dizi:** İstenen göze doğrudan erişilir. Örneğin `A[2] = 5`. Sıralama, değer değiştirme, ekleme ve silmede bu rastgele erişim gerekir.
- **Bağlı liste:** Elemanlara sırayla erişilir. Ortadaki bir elemana doğrudan atlanamaz; baştan veya sondan adım adım gidilir.
- **Yığın:** Yalnız en son eklenen elemanla işlem yapılır.

Yığın (Stack) ve LIFO

Örnekler:

- **Undo/redo:** Hoca bir metin editöründe "kapı" yazıp undo yaptı. Harfler son yazılandan başlayarak sırayla silindi (ı, p, a, k). Redo ile aynı sırayla geri geldi.
- **Tarayıcıda geri (back):** Gazete sayfası → yazarlar → bir yazının sayfası. Her geri, en son gelinen sayfaya döner.
- **Fonksiyon çağrıları:** `main` → `f1` → `f2`. `f2` bitince `f1`'de kalınan yere, `f1` bitince `main`'de kalınan yere dönülür.
- **İfade değerlendirme (expression evaluation):** Derleyici aritmetik ifadeleri yığın kullanarak işlem önceliğine göre çözer.

Görsel olarak yığın, dibi 0. adres olan ve yukarı doğru dolan bir bardak gibi düşünülebilir:

```
push A, push B, push C       x = pop   ->  x = C
|   |                        |   |
| C | <- top                 |   |
| B |                        | B | <- top
| A |                        | A |
+---+                        +---+
```

Dizi ile Yığın Gerçekleştirme

```c
#define MAX 100

typedef struct {
    int items[MAX];
    int top;
} stack;

void initialize(stack *s)
{
    s->top = 0;
}

int isEmpty(stack *s)
{
    if (s->top == 0)
        return 1;
    else
        return 0;
}

int isFull(stack *s)
{
```

```
    if (s->top == MAX)
        return 1;
    else
        return 0;
}

int push(int value, stack *s)
{
    if (isFull(s)) {
        printf("stack is full");
        return 0;
    }
    s->items[s->top] = value;
    s->top++;
    return 1;
}

int pop(int *value, stack *s)
{
    if (isEmpty(s))
        return 0;
    s->top--;
    *value = s->items[s->top];
    return 1;
}
```

- `top`, bir sonraki elemanın yazılacağı gözü gösterir.
- Hoca `top`'u -1'den başlatıp önce artırma, sonra yazma yönteminin de mümkün olduğunu, ikisi arasında ciddi fark olmadığını söyledi.
- **`isFull` örneği:** `MAX = 3`, gözler 0, 1, 2. 5, 3, 7 push edilince `top = 3` olur. 3 numaralı göz diziye ait değildir; yığın doludur.
- **`push`:** Doluysa 0 döndürür (veya -1, tercihe bağlı). Değilse değeri `items[top]`'a yazar, `top`'u artırır ve 1 döndürür.
- **`pop`:** Boşsa 0 döndürür. Değilse önce `top`'u azaltır, sonra o gözdeki değeri `value`'ya yazar ve 1 döndürür.
  - Değer, adresi gönderilen `value` değişkeninde döner. Dönüş değeri yalnız başarıyı gösterir.
  - Alternatif: Yığına yalnız pozitif değerler konacağı biliniyorsa, başarısızlıkta -1, başarıda değerin kendisi döndürülebilir.
- **`peek`:** `pop` gibi en üstteki elemanı verir ama `top`'u değiştirmez. En üstteki eleman `items[top - 1]`'dedir. Bu fonksiyon öğrencilere bırakıldı.

Aritmetik İfade Değerlendirme

Neden yığın? `7 + 5 * 4 / 3 * 2 - 8` gibi bir ifadede önce `5 * 4`, sonra `/ 3`, sonra `* 2`, sonra `7 +`, en son `- 8` yapılır. `7 +` işlemi uzun süre "akılda tutulmalıdır". Bu yığına çok uygun bir yapıdır.

İşlem öncelikleri (yalnız tüm dillerde ortak olan temel işlemler):

Durum	En yüksekten en düşüğe
Push önceliği (yığına girerken)	`(` > `*`, `/` > `+`, `-`
Pop önceliği (yığından çıkarken)	`*`, `/` > `+`, `-` > `(`

- Yeni gelen işlemci, yığının üstündeki işlemciyle karşılaştırılır. Üstteki işlemci eşit veya daha yüksek öncelikliyse çıkarılır ve işlemi yapılır.
- `(` girerken en yüksek önceliklidir, hiçbir işlemciyi çıkarmaz. Yığındayken en düşük önceliklidir; parantez içindekiler bitmeden çıkmaz.
- `)` yığına konmaz. `(` görülene kadar yığındaki tüm işlemciler çıkarılıp işlenir; sonra `(` atılır.
- İfade bitince yığında kalan her şey sırayla çıkarılıp işlenir.

- İki işlenen çekildiğinde işlem **alttaki (işlem) üstteki** şeklinde yapılır.

Örnek 1: `x = 3 + 5 * 8 - 2` (iki yığınla)

Gelen	İşlenen yığını	İşlemci yığını	Açıklama
3	3		
+	3	+	
5	3, 5	+	
*	3, 5	+, *	+ daha düşük öncelikli, kalır
8	3, 5, 8	+, *	
-	3, 40	+	* daha öncelikli: İşlem 1, $5 \cdot 8 = 40$
	43	-	+ eşit öncelikli: İşlem 2, $3 + 40 = 43$; - yığına girer
2	43, 2	-	
;	41		Bitti: İşlem 3, $43 - 2 = 41$

Örnek 2: `3 * (5 + 8 / 2) - 4` (iki yığınla)

Gelen	İşlenen yığını	İşlemci yığını	Açıklama
3	3		
*	3	*	
(	3	*, (	(push'ta en öncelikli
5	3, 5	*, (	
+	3, 5	*, (, +	(pop'ta en düşük, çıkmaz
8	3, 5, 8	*, (, +	
/	3, 5, 8	*, (, +, /	+ daha düşük, kalır
2	3, 5, 8, 2	*, (, +, /	
)	3, 5, 4	*, (, +	İşlem 1: $8/2 = 4$
	3, 9	*, (	İşlem 2: $5 + 4 = 9$
	3, 9	*	(atılır
-	27	-	* daha öncelikli: İşlem 3, $3 \cdot 9 = 27$
4	27, 4	-	
son	23		İşlem 4: $27 - 4 = 23$

Infix'ten Postfix'e Çevirme Yalnız işlemci yığını kullanılır. İşlenenler geldikleri sırada doğrudan çıktıya yazılır.

`3 * (5 + 8 / 2) - 4` için:

Gelen	Çıktı	İşlemci yığını
3	3	
*	3	*
(	3	*, (
5	3 5	*, (
+	3 5	*, (, +
8	3 5 8	*, (, +
/	3 5 8	*, (, +, /
2	3 5 8 2	*, (, +, /
)	3 5 8 2 / +	*
-	3 5 8 2 / + *	-
4	3 5 8 2 / + * 4	-

Gelen	Çıktı	İşlemci yığını
son	3 5 8 2 / + * 4 -	

Sonuç: `3 5 8 2 / + * 4 -`.

Postfix İfadeyi Çözme Yalnız işlenen yığını kullanılır. İşlenen gelince yığına konur. İşlemci gelince üstteki iki değer çekilir, işlem yapılır, sonuç yığına konur.

Gelen	Yığın	İşlem
3, 5, 8, 2	3, 5, 8, 2	
/	3, 5, 4	$8/2 = 4$
+	3, 9	$5 + 4 = 9$
*	27	$3 \cdot 9 = 27$
4	27, 4	
-	23	$27 - 4 = 23$

- İki yöntem:
  1. Infix ifadeyi iki yığınla tek seferde çözmek (derleyicinin yaptığı).
  2. Önce işlemci yığınıyla postfix'e çevirmek, sonra işlenen yığınıyla postfix'i çözmek.
- Hoca kodunu yazmadı. Kodun `push`, `pop`, `isEmpty`, `isFull` ile rahatça yazılabileceğini söyledi.

Kuyruk (Queue) ve FIFO

- Market veya banka kuyruğu gibi: İlk gelene ilk hizmet verilir.
- Örnekler:
  - Bilgisayar yavaşken basılan tuşlar ekranda basılma sırasıyla görünür.
  - Yazıcıya gönderilen işler sırayla basılır.
  - CPU zamanlaması (CPU scheduling), giriş/çıkış (I/O) işlemleri.
- Yalnız kuyruğun başındaki elemana erişilmesi gerekiyorsa, ortadaki elemanlara erişilmemesi gerekiyorsa kuyruk kullanılır.
- İşlemler: `enqueue` (sona ekle), `dequeue` (baştakini çıkar).

Basit Kuyruk

```
MAX = 5
index:  0   1   2   3   4
       [3] [5] [7] [8] [ ]
front = 0         rear = 3
```

- `enqueue(9)`: 9, 4. göze yazılır; `rear = 4`.
- `x = dequeue()`: `x = 3`. Diğer elemanlar birer öne kaydırılır. `front` hep 0'dır.
- `enqueue` $O(1)$, `dequeue` $O(n)$'dir. Çok hareketli kuyruklarda bu kaydırma istenmez.

Dairesel Kuyruk (Circular Queue)

- Eleman çıkınca diziyi kaydırmak yerine `front` bir ilerletilir.
- Yeni eleman sona eklenir; dizinin sonuna gelindiyse başa dönülür.
- `enqueue` ve `dequeue` $O(1)$ olur.
- Örnek: `9, 5, 2, 1, 4` (front 5'te, rear 9'da). `x = dequeue()` ile `x = 5`, `front` 2'yi gösterir. 5 fiziksel olarak dizide durur ama artık kullanılabilir bir gözdür. `enqueue(7)` ile 7, 5'in yerine yazılır ve `rear` orayı gösterir.

```c
typedef struct {
    int items[MAX];
    int front, rear;
} queue;
```

```c
void initialize(queue *q)
{
    q->front = -1;
    q->rear = -1;
}

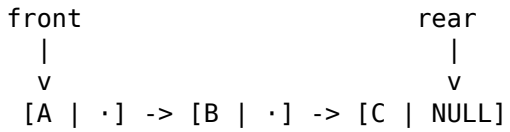
int isEmpty(queue *q)
{
    if (q->front == -1)
        return 1;
    else
        return 0;
}

int isFull(queue *q)
{
    if ((q->front == 0 && q->rear == MAX - 1) || (q->front == q->rear + 1))
        return 1;
    else
        return 0;
}

int enqueue(queue *q, int value)
{
    if (isFull(q)) {
        printf("queue is full");
        return 0;
    }
    if (q->front == -1)              /* yalnız ilk elemanda */
        q->front = 0;
    q->rear = (q->rear + 1) % MAX;
    q->items[q->rear] = value;
    return 1;
}
```

- Boşluk için yalnız `front`'u kontrol etmek yeterlidir; `front` ve `rear` boşken birlikte -1'dir.
- Doluluğun iki durumu:
  - Hiç eleman çıkarılmamış: `front = 0` ve `rear = MAX - 1`.
  - Dönüş olmuş: `front = rear + 1`. Örneğin dizi `[5, 7, 2, 3]`, `rear` 1'de ve `front` 2'deyse kuyruk doludur; işlem sırası `2, 3, 5, 7`'dir.
  - `front > rear` ama aralarında boş göz varsa kuyruk dolu değildir.
- `enqueue`'da `front` yalnız kuyruk boşken bir kez 0 yapılır; sonra eklemede `front`'a dokunulmaz.
- `rear = (rear + 1) % MAX`: Örneğin `MAX = 4`, `rear = 3` iken yeni eleman $(3+1) \bmod 4 = 0$ numaralı göze yazılır.
- `dequeue` öğrencilere bırakıldı. İpucu: `front`'taki değer alınır, `front` bir ilerletilir; dönüş değeri `pop`'taki gibi başarıyı gösterebilir.

Bağlı Liste ile Kuyruk



- `dequeue`: `front` bir sonraki düğüme geçer.
- `enqueue`: Yeni düğüm sona bağlanır, `rear` ona geçer.
- Dolu olma durumu yoktur; yalnız `malloc` yer ayıramazsa hata olur.

```c
typedef struct qnode {
```

```
    int value;
    struct qnode *next;
} qnode;

typedef struct {
    qnode *front, *rear;
} queue;

queue *createQueue()
{
    queue *q = (queue *) malloc(sizeof(queue));
    q->front = NULL;
    q->rear = NULL;
    return q;
}

qnode *newNode(int newValue)
{
    qnode *temp = (qnode *) malloc(sizeof(qnode));
    if (temp == NULL)
        exit(1);
    temp->value = newValue;
    temp->next = NULL;
    return temp;
}

void enqueue(queue *q, int value)
{
    qnode *temp = newNode(value);
    if (q->front == NULL) {          /* kuyruk boş */
        q->front = temp;
        q->rear = temp;
    }
    else {
        q->rear->next = temp;
        q->rear = temp;
    }
}

int dequeue(queue *q)
{
    qnode *temp;
    int value;
    if (q->front == NULL) {
        printf("queue is empty");
        return -1;
    }
    temp = q->front;
    value = temp->value;
    q->front = q->front->next;
    free(temp);
    if (q->front == NULL)            /* kuyruk boşaldı */
        q->rear = NULL;
    return value;
}
```

- **enqueue örneği:** `front = 1500 [9], 1700 [8], rear = 1800 [17]`. Yeni düğüm 1100: `1800->next = 1100`, `rear = 1100`. `front` değişmez.
- Boş kuyruğa ilk eleman eklenince hem `front` hem `rear` o düğümü gösterir.
- **dequeue örneği:** `front = 1000 [8] -> 1100 [9] -> 1200 [5]`, `rear = 1200`.

  - `value = 8`, `front = 1100`, `free(1000)`.
- **Son eleman çıkınca:** Kuyrukta yalnız 1000 [8] varken çıkarılırsa `front` `NULL` olur ama `rear` hâlâ 1000'i gösterir. Bu yüzden `front == NULL` ise `rear` da `NULL` yapılmalıdır.
- Bir öğrenci `qnode` ile `queue` tiplerini karıştırdı. `qnode` kuyruğun her düğümüdür; `queue` ise yalnız `front` ve `rear` pointer'larını tutan yapıdır.
- `isEmpty` ayrı bir fonksiyon olarak öğrencilere bırakıldı.

Soru: Yığındaki En Büyük Eleman (Max Stack)

- Yalnız `push`, `pop` ve `peek` kullanılabiliyor. Yığındaki en büyük eleman her an bilinmeli.
- Örnek: 3, 1, 7, 2, 9, 5 push edilir; en büyük sırayla 3, 3, 7, 7, 9, 9 olur. 5 ve 9 pop edilince en büyük 7 olmalıdır.
- Tek bir `max` değişkeni yetmez: 9 çıkınca öncekini bilemezsiniz.
- İki değişken (`max` ve önceki `max`) de yetmez: 7 de çıkarsa 3'ü bilemezsiniz.
- Aradaki elemanlara başka bir pointer ile erişmek yasaktır.
- **Çözüm (bir öğrenci önerdi, hoca onayladı):** En büyük değerleri tutan ikinci bir yığın (max stack) kullanılır.
  - Push edilen değer o anki en büyükten büyükse max yığınına da push edilir.
  - Pop edilen değer max yığınının üstündekine eşitse max yığınından da pop edilir.
  - En büyük eleman her zaman max yığınının üstündedir. Hızlı erişim için ayrıca bir `max` değişkeni de tutulabilir.

İşlem	Yığın	Max yığını	En büyük
push 3	3	3	3
push 1	3, 1	3	3
push 7	3, 1, 7	3, 7	7
push 5	3, 1, 7, 5	3, 7	7
push 9	3, 1, 7, 5, 9	3, 7, 9	9
push 11	…, 9, 11	3, 7, 9, 11	11
push 2	…, 11, 2	3, 7, 9, 11	11
pop 2	…, 11	3, 7, 9, 11	11
pop 11	…, 9	3, 7, 9	9
pop 9	3, 1, 7, 5	3, 7	7
pop 5	3, 1, 7	3, 7	7
pop 7	3, 1	3	3

Ders sonrası notlar

- **Kuyruk / yığın terimleri:** Hoca kuyruğu tanıtırken "queue, yığın veri yapısı" dedi. Doğrusu: stack = yığın, queue = kuyruk.
- **Dairesel kuyrukta doluluk kontrolü:** Derste yazılan `front == rear + 1` koşulu, `front == 0` ve `rear == MAX - 1` durumunu ayrıca kontrol ettiği için doğru çalışır. Bu iki koşul tek ifadeyle `(rear + 1) % MAX == front` olarak yazılabilir.
- **Bağlı liste kuyruğunda `dequeue`:** Hoca bir yerde "çıkacak eleman kuyruğun en sonundaki eleman" dedi, ardından kodu kuyruğun başındaki (front) elemanı çıkaracak şekilde yazdı. Doğrusu: `dequeue` her zaman kuyruğun başındaki elemanı çıkarır.
- **Max stack'te eşit değerler:** Derste max yığınına yalnız "büyük" değerler kondu. Aynı en büyük değer iki kez push edilirse (örneğin 9, 9), birincisi pop edilince max yığınından da 9 çıkar ve en büyük yanlış görünür. Bu yüzden push'ta koşul "büyük veya eşit" (`>=`) olmalıdır.
- **Önceliklerde eşitlik:** Örnek 1'de `-` geldiğinde yığındaki `+` eşit öncelikli olduğu için çıkarıldı. Bu, aynı öncelikli işlemlerin soldan sağa yapılmasını sağlar. `+`, `-`, `*`, `/` için kural "üstteki eşit veya daha öncelikliyse çıkar" şeklindedir.
- **Postfix çözümünde atlanan adım:** Hoca derste önce 2'yi postfix ifadeye yazmayı unuttu, sonra çözümde 3'ü yığında bıraktı; ikisini de kendisi fark edip düzeltti. Yukarıdaki tablolar düzeltilmiş hâlidir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.